perf_capture.sh 使用指南

Oct 14, 2026 · @john.he

`perf_capture.sh` 在开发板上一键完成 perf
采集：自动设置内核参数、采集、导出内核符号和文本、调用 perf_analyzer
生成报告，每次结果放在单独的目录里，支持定时采集和自动清理。

1. 概述与依赖

项目	说明
运行位置	开发板（E3、SS 都适用），兼容 BusyBox sh
运行权限	root（需要修改内核参数、读取内核符号）
必需	与板子内核匹配的 perf（E3 用 64 位版本，SS 用 32 位版本）
可选	perf_analyzer：存在时自动生成分析报告、地址清单和折叠栈；不存在时只保存原始数据
输出位置	默认 `/data/perfwork`，应放在持久数据分区，不要用 `/tmp` 等内存文件系统

2. 部署

```
# 主机上
scp perf_capture.sh root@<板子IP>:/data/

# 板子上
chmod +x /data/perf_capture.sh
which perf perf_analyzer        # 确认工具已部署
/data/perf_capture.sh -h        # 查看帮助
```

perf 或 perf_analyzer 不在 PATH 中时，可以用环境变量指定路径：`PERF=/data/bin/perf ANALYZER=/data/bin/perf_analyzer /data/perf_capture.sh`。

3. 快速开始

按位置依次传入**进程名、采样秒数、采样频率**，后面的参数可以省略：

```
/data/perf_capture.sh ApplicationServices 60 199     # 采 ApplicationServices，60 秒，199 Hz
/data/perf_capture.sh ApplicationServices 60         # 频率用默认 199 Hz
/data/perf_capture.sh                                #
全部用默认：ApplicationServices，120 秒，199 Hz
```

运行时会打印类似下面的信息：

```
[10:20:01] 已设置 perf_event_paranoid=-1 kptr_restrict=0（原值 2/0，退出时恢复）
[10:20:01] 进程 ApplicationServices，每次 60 秒，频率 199 Hz，共 1 次
[10:20:01] 输出目录 /data/perfwork，剩余 2400 MB；perf_analyzer：可用
[10:20:01] 第 1/1 次：ApplicationServices (pid 3570) 采集 60 秒 ->
/data/perfwork/ApplicationServices_20261014_102001
[10:21:03] 分析报告: /data/perfwork/ApplicationServices_20261014_102001/report.txt
[10:21:03] 完成，占用 3120 KB，剩余空间 2397 MB
```

采集结束后，在输出目录里直接查看 `report.txt`，或按第 6 节把文件拷回主机做进一步分析。

4. 参数说明

完整格式：`perf_capture.sh [选项] [进程名 [采样秒数 [采样频率]]]`。选项写在前面，位置参数写在最后；两种写法同时出现时，以位置参数为准。

4.1 位置参数

位置	含义	默认值
第 1 个	进程名（与 `pidof` 使用的名字一致）	ApplicationServices
第 2 个	每次采集秒数	120
第 3 个	采样频率（Hz）	199

4.2 选项

选项	作用	默认值
`-p 进程名`	要采集的进程（同第 1 个位置参数）	ApplicationServices
`-t 秒数`	每次采集时长（同第 2 个位置参数）	120
`-F 频率`	采样频率 Hz（同第 3 个位置参数）	199
`-e 事件`	采样事件	cpu-clock
`-m 页数`	环形缓冲区页数，需为 2 的幂；出现丢样本时加大	64
`-o 目录`	输出目录	/data/perfwork
`-n 次数`	采集次数	1

选项	作用	默认值
`-i 秒数`	多次采集时的间隔，按每次开始时间计算	0（连续）
`-k 个数`	只保留最近 N 次结果，旧的自动删除	0（不清理）
`-f MB`	输出目录剩余空间低于该值时跳过本次采集	200
`-a`	全系统采集，不指定进程	—
`-S`	只用 perf stat 计数（CPU 时间、上下文切换、缺页），不采样，几乎无开销	—
`-N`	不保存 `out.perf.gz`，节省空间	—
`-z`	每次采集后把结果目录打包成 tar.gz	—
`-h`	显示帮助	—

4.3 环境变量

变量	作用	默认值
`PERF`	perf 命令路径	perf
`ANALYZER`	perf_analyzer 命令路径	perf_analyzer

参数写错时脚本会直接报错退出，例如秒数写成 `1x` 会提示"DURATION 必须是非负整数"，多给参数会提示"多余的参数"。

5. 常用场景

场景	命令
问题出现时采 1 分钟	`/data/perf_capture.sh ApplicationServices 60`
采集其他进程	`/data/perf_capture.sh DataServer 120 199`
每小时采 2 分钟，共 24 次，只保留最近 12 次	`/data/perf_capture.sh -n 24 -i 3600 -k 12 ApplicationServices 120`
长时间观察趋势（只计数，几乎无开销）	`/data/perf_capture.sh -S ApplicationServices 600`

场景	命令
全系统采集（看中断、内核线程、其他进程）	`/data/perf_capture.sh -a -t 30`
采完打包，方便拷贝	`/data/perf_capture.sh -z ApplicationServices 120`
空间紧张时不保存文本	`/data/perf_capture.sh -N ApplicationServices 120`

后台长时间运行：断开 SSH 后继续执行，日志写到文件：

```
nohup /data/perf_capture.sh -n 24 -i 3600 -k 12 ApplicationServices 120 \
    > /data/perfwork/capture.log 2>&1 &
tail -f /data/perfwork/capture.log      # 查看进度
```

中途停止：前台运行时按 Ctrl+C；后台运行时 `kill <脚本进程号>`。脚本会正常结束 perf、保存已采集的数据，并恢复内核参数。

6. 输出文件与后续分析

每次采集生成一个目录 `<输出目录>/<进程名>_<日期>_<时间>/`（全系统采集时为 `system_<日期>_<时间>`）：

文件	内容	用途
`info.txt`	采集时间、内核、perf 版本、程序路径和 **md5**、采集命令、样本统计	主机上换算地址前核对程序版本
`report.txt`	perf_analyzer 分析报告（板子上有 perf_analyzer 时）	直接在板子上阅读
`addrs.txt`	地址清单	拷到主机用 `--resolve-addrs` 换算函数名
`out.folded`	折叠栈	拷到主机用 `flamegraph.pl` 画火焰图
`out.perf.gz`	带内存映射的 perf script 文本	拷到主机用 perf_analyzer `--symfs` 完整分析
`perf.data`	perf 原始数据	需要时用 perf 重新解析
`kallsyms`	内核符号表	主机上用 perf 解析时配合 `--kallsyms`
`maps.txt`	采集前的进程内存映射快照	备用，配合 perf_analyzer `--maps`

文件	内容	用途
record.log、analyzer.log	perf 和 perf_analyzer 的运行日志	排查失败原因
stat.txt	只计数模式（-S）的输出	查看 CPU 时间、切换次数等随时间的变化

6.1 拷回主机

```
# 主机上（加了 -z 时直接拷 tar.gz）
mkdir -p /home/hejr/perfwork && cd /home/hejr/perfwork
scp -r root@<板子IP>:/data/perfwork/ApplicationServices_20261014_102001 .
cd ApplicationServices_20261014_102001
```

6.2 核对版本

用 info.txt 中记录的 md5，与主机上同一次编译的程序比对（SS 用 rootfs_newarm，E3 用 rootfs_arm64-short）：

```
STRIP=/opt/arm-buildroot-linux-gnueabihf_sdk-buildroot/bin/arm-buildroot-linux-
gnueabihf-strip
ROOTFS=/home/hejr/workspace/facsplat_sw/G4_small/rootfs_newarm
$STRIP --strip-all -o /tmp/check
$ROOTFS/rootfs_partition_unstripped/opt/ApplicationServices/ApplicationServices
md5sum /tmp/check
grep md5 info.txt
```

两个值一致，才能用这份符号目录换算。

6.3 换算函数名、完整分析、画火焰图

```
SYMFS=$ROOTFS/rootfs_partition_unstripped

# 只拷了 addrs.txt：把地址换算成函数名
perf_analyzer --resolve-addrs addrs.txt --symfs $SYMFS > funcs.txt

# 拷了 out.perf.gz：带完整函数名的报告和折叠栈
gunzip -f out.perf.gz
perf_analyzer out.perf --duration 120 --symfs $SYMFS --folded-out full.folded >
full_report.txt

# 画火焰图
/home/hejr/FlameGraph/flamegraph.pl --title "ApplicationServices" full.folded >
flame.svg
```

详细说明见《perf_analyzer 使用说明书》第 8、9 节。

7. 脚本的工作流程与保护机制

7.1 工作流程

1. 检查参数、root 权限、perf 是否存在，创建输出目录。
2. 保存内核参数原值，然后设置：

```
echo -1 > /proc/sys/kernel/perf_event_paranoid   # 允许采集所有事件和内核调用栈
echo 0  > /proc/sys/kernel/kptr_restrict         # 让 /proc/kallsyms
显示真实地址
```

3. 每次采集：检查剩余空间 → `pidof` 获取进程号 → 写 `info.txt`、保存 `maps.txt` → 执行 perf record（或 `-S` 时执行 perf stat）。
4. 采集结束后立即导出 `kallsyms`，统计样本数，执行 `perf script --show-mmap-events`。
5. 有 perf_analyzer 时生成 `report.txt`、`addrs.txt`、`out.folded`；压缩 `out.perf`。
6. 按需打包（`-z`）、清理旧结果（`-k`），等待到下一次采集的开始时间。
7. 全部结束（或中断）后恢复内核参数。

实际执行的采集命令为：

```
perf record -e cpu-clock -F <频率> -m 64 --call-graph fp --proc-map-timeout 10000 \
    -p <进程号> -o perf.data -- sleep <秒数>
```

7.2 保护机制

情况	脚本的处理
按 Ctrl+C 或被 kill	向 perf 发送中断信号让它正常写完数据，然后恢复内核参数
剩余空间不足	低于 `-f` 指定值（默认 200 MB）时跳过本次采集
进程不存在	跳过本次采集并提示，定时采集时下一次继续尝试
进程重启	每次采集前重新获取进程号，定时采集不受影响
同名进程有多个	只采集第一个，并提示所有进程号
perf record 失败	提示错误并保留 `record.log`，不影响后续轮次
perf_analyzer 不存在或失败	只保存原始数据；失败原因写入 `analyzer.log`
结果越来越多	`-k` 只保留最近 N 次，旧目录和对应的 tar.gz 一并删除

8. 常见问题

现象	原因	处理
请用 root 运行	当前不是 root 用户	切换到 root 再执行
找不到 perf	perf 不在 PATH 中	部署 perf，或用 PERF=/路径/perf 指定
perf_analyzer：未找到，只保存原始数据	板子上没有 perf_analyzer	正常，可以拷回主机分析；需要板子上出报告时部署 perf_analyzer
找不到进程 xxx	进程名写错或进程未运行	用 ps 确认名字，进程名与 pidof 使用的一致
无法设置 /proc/sys/kernel/...	不是 root，或内核不允许修改	用 root 执行；仍失败时内核符号可能显示为 [unknown]
perf record 失败	perf 与内核不匹配、事件不支持等	查看结果目录中的 record.log；E3 用 64 位 perf，SS 用 32 位 perf
剩余 xx MB，低于 xx MB，跳过本次采集	输出目录空间不足	清理旧结果、换输出目录（-o），或调低 -f
info.txt 中有 lost 记录	环形缓冲区溢出，丢了样本	加大 -m（如 128），或降低频率
report.txt 中业务函数只有地址	板子上的程序被 strip	正常，按第 6 节拷回 addrs.txt 在主机上换算
采集结束后还想手动运行 perf	脚本退出时已恢复内核参数	手动执行 echo -1 > /proc/sys/kernel/perf_event_paranoid 和 echo 0 > /proc/sys/kernel/kptr_restrict

相关文档：《perf_analyzer 使用说明书》、《perf 长时间采集的资源占用与建议》。